Intelligent USB Device Behaviour Monitoring System (IUDBMS)

A Behaviour-Based Approach to Runtime USB Peripheral Trust

1. Introduction

USB has been the dominant peripheral standard since 1996, prized for plug-and-play convenience. That convenience rests on a persistent liability: USB trusts devices by default. During enumeration a device self-reports its Vendor ID (VID), Product ID (PID), and interface descriptors, and the host accepts these claims without cryptographic attestation.

This trust-by-default model is exploited by **BadUSB** attacks, where firmware is altered to declare an interface it does not honestly represent — typically a Human Interface Device (HID) keyboard — so an object resembling an ordinary flash drive or cable can inject a pre-programmed keystroke payload the moment it is plugged in. Because HID input is inherently trusted, these attacks bypass file-based antivirus and execute within seconds.

Existing defences largely verify device *identity* at the point of connection — allowlisting by VID/PID, user role-confirmation, or static kernel filter rules. None of these judge how a device *actually behaves* once operating, so a device that spoofs an allowed identity, or misbehaves within an approved role, defeats them entirely.

This project proposes the **Intelligent USB Device Behaviour Monitoring System (IUDBMS)**, which establishes *behavioural trust at runtime* rather than identity at connection time. IUDBMS captures USB Request Blocks (URBs) at the kernel/driver boundary using eBPF, scores each device against a per-device adaptive baseline, models the temporal sequence of its events with an LSTM to catch staged or slow-burn attacks, and enforces verdicts through a closed loop inside the kernel — rate-limiting or unbinding a malicious device before the attack window closes.

The remainder of this document reviews the literature across three themes — USB/BadUSB defences, eBPF kernel observability, and LSTM-based anomaly detection — then derives the problem statement, research gap, novelty, and objectives of IUDBMS.

2. Literature Review

The relevant literature falls into three strands: the USB attack surface and defences against malicious peripherals; eBPF as a mechanism for kernel-level observation and enforcement; and deep-learning methods for temporally-structured anomaly detection. IUDBMS sits at their intersection.

2.1 USB Peripheral Security and BadUSB Defences

Shen et al. [1] proposed **WMUD**, a whitelist method for USB redirection in virtual-desktop infrastructure (QEMU/SPICE). Server-side filter rules — keyed on device class, VID, and PID — admit only permitted devices and cannot be tampered with from the client. This shows the value of protected, per-asset policy, but remains a pure identity-based allowlist: a device presenting an allowed identifier is admitted

regardless of subsequent behaviour, and the scheme is scoped to virtualised redirection rather than direct host defence.

Bintang et al. [2] proposed **SafeUSB**, a Windows keystroke monitor combining an inter-keystroke timing threshold with content analysis that flags typed keywords such as *POWERSHELL* and *CMD.EXE*. It blocks most injection scenarios, but the authors report a critical ~1-second delay between HID connection and monitoring activation, leaving short payloads a window to succeed — and recommend, as future work, moving to compiled, kernel-level hooks, precisely the direction IUDBMS takes.

Together [1], [2] these works frame the tension IUDBMS resolves: WMUD's tamper-resistant, per-asset policy is identity-bound, while SafeUSB's behavioural analysis is limited by userspace placement and activation delay. Neither builds an adaptive per-device model, and neither enforces at the kernel/driver boundary.

2.2 eBPF for Kernel-Level Observability and Enforcement

eBPF allows verified bytecode to run inside the Linux kernel, giving userspace-authored logic fine-grained observability and enforcement at kernel speed. Song's survey [5] traces its core mechanics — the in-kernel VM, JIT compilation, and the static verifier that guarantees bounded, memory-safe execution — and its expansion from packet filtering into tracing and security, while noting persistent constraints around verification complexity and program-size limits.

Kostopoulos et al. [3] presented **eIDPS**, a real-time eBPF-plus-ML network intrusion detection and prevention pipeline operating at line rate. Rajurkar and Deshmukh [4] survey the field's architectural patterns and identify the dominant one: *hybrid designs that extract features in the kernel and delegate ML inference to userspace*, coordinated through shared BPF maps — alongside open challenges in verifier constraints, portability, and ML integration.

This body of work [3]–[5] establishes eBPF as a proven substrate for sub-microsecond observation and enforcement, and the kernel-extract/userspace-infer split as the established way to pair kernel speed with ML expressiveness. But it concerns *network* intrusion detection; applying the same pattern to the *USB* subsystem — URBs as the unit of observation, the driver-bind boundary as the enforcement point — is unexplored.

2.3 Deep Learning for Behavioural and Temporal Anomaly Detection

LSTM networks are well suited to anomalies whose signal lies in temporal structure rather than any single point in time. Hnamte et al. [7] proposed **LSTM-AE**, hybridising an LSTM with an auto-encoder for network intrusion detection, validated across multiple public datasets and arguing for two-stage designs that separate detection from classification.

Baek and Jung [6] similarly decouple anomaly detection from attack classification: a binary focal-loss stage filters anomalies from heavily imbalanced traffic, and a second multi-class stage identifies the attack type, both sharing a CNN-1D-LSTM backbone — a staged decomposition IUDBMS's own layering parallels. Saini et al. [8] reinforce the value of hybrid *CNN–LSTM* architectures for network threat detection.

Two further works target novel and slow-developing attacks directly. Chithik Raja and Al Mahri [9] applied an **attention-enhanced LSTM** to zero-day detection across process, network, and file-system telemetry, showing behaviour-based temporal modelling generalises to unseen threats. Mohamed Anwar et al. [10] applied **LSTM-powered anomaly detection** to IIoT systems against DDoS, spoofing, and ransomware, outperforming Random Forest, Logistic Regression, and 1D-CNN baselines at low latency.

Collectively [6]–[10], this strand shows LSTM-based temporal models catch multi-stage and zero-day behaviour within real-time budgets, and that layered designs balance recall against discrimination. What it does not do is apply temporal modelling to a single USB peripheral's event stream — the application IUDBMS pursues.

3. Inferences from the Literature

The USB-defence literature [1], [2] establishes the threat and the value of behavioural analysis but stays tied to identity or userspace latency. The eBPF literature [3]–[5] establishes a fast, verifiable kernel-resident mechanism, almost exclusively for networks. The deep-learning literature [6]–[10] establishes that layered LSTM models catch temporally-structured, zero-day behaviour in real time, but never on the USB event stream. IUDBMS sits in the space these three strands leave open.

3.1 Problem Statement

USB's trust-by-default enumeration lets a malicious peripheral (BadUSB) declare a trusted identity — typically an HID keyboard — and abuse it to inject commands at machine speed, compromising the host within seconds. Existing defences authenticate *claimed identity* at connection but do not continuously judge *actual runtime behaviour*. Devices that spoof allowed identifiers, or misbehave within an approved role, evade detection; and existing behavioural monitors run in userspace, incurring an activation delay that lets short payloads succeed. A defence is needed that (a) trusts devices by observed behaviour, not declared identity, (b) enforces at the kernel/driver boundary to remove round-trip latency, and (c) detects staged, slow-escalation attacks, not just single-point anomalies.

3.2 Research Gap

- **Identity-bound trust:** Allowlisting (e.g., WMUD [1]) admits devices by class/VID/PID and is blind to spoofed identities or misuse within an approved role.
- **Userspace activation latency:** SafeUSB [2] proves runtime analysis works, but its userspace placement adds a ~1-second activation delay that short payloads exploit.
- **Network-only kernel intelligence:** The eBPF-plus-ML pattern (eIDPS [3]; Rajurkar & Deshmukh [4]) is mature for networks but unapplied to USB, where the natural unit is the URB and the natural enforcement point is driver-bind.
- **No temporal modelling of USB behaviour:** LSTM/attention models [6]–[10] catch staged, zero-day attacks in network and IIoT domains, but no USB defence models a device's event sequence over time.

- **Open enforcement loop:** Existing behavioural detectors alert or block from userspace after a round-trip; none close the loop inside the kernel.

3.3 Novelty of IUDBMS

Individually, the underlying techniques — eBPF capture [3]–[5], unsupervised anomaly scoring, and LSTM temporal modelling [6]–[10] — already exist. IUDBMS's novelty is the *combination and application*: fusing them into one per-device, closed-loop pipeline and pointing that pipeline at the USB subsystem, which no reviewed eBPF-ML or LSTM-anomaly work addresses and no reviewed USB-defence work implements. Each element below answers a gap from Section 3.2:

1. **Multi-modal feature fusion at the kernel/driver boundary.** Fuses event timing, traffic volume, and descriptor changes into one feature stream captured via eBPF — carrying the eBPF-ML pattern [3], [4] into USB, answering the network-only gap.
2. **Per-device adaptive behavioural baselines.** Each device is scored against its own learned norm rather than a global rule set, answering the identity-bound trust gap left by WMUD [1].
3. **Closed-loop kernel-level enforcement.** Verdicts (rate-limiting, unbinding) execute inside the kernel with no userspace round-trip, answering SafeUSB's [2] activation-latency limitation and the open-enforcement-loop gap.
4. **Temporal sequence modelling to catch staged attacks.** A userspace LSTM analyses rolling event windows to catch slow-escalation attacks single-point detectors miss, bringing temporal modelling [6]–[10] to USB for the first time.

3.4 Relationship to the Reference Framework

The reference framework is a kernel-level eBPF security framework that filters peripherals against human-authored rules at the host-controller boundary with sub-microsecond overhead. IUDBMS keeps its core insight — the kernel/driver boundary, mediated by eBPF, is the right place to enforce — but replaces its static, administrator-written rules with *dynamic behavioural intelligence*: per-device adaptive baselines and anomaly scoring in the kernel fast-path, backed by a userspace LSTM. In short, IUDBMS keeps the reference framework's fast, verifiable enforcement substrate and adds the adaptive, behaviour-learning capability it explicitly does not attempt — moving from *rules an administrator must foresee* to *behaviour the system learns on its own*. (No implementation work is done against the reference framework; it is discussed only to position IUDBMS's contribution.)

4. Objectives

The aim is to build a system that trusts USB peripherals by runtime behaviour rather than claimed identity, via a four-layer pipeline carrying a device's activity from raw kernel events, through in-kernel scoring, to userspace temporal analysis, and back into the kernel as an enforced verdict — following the kernel-extract/userspace-infer pattern [3], [4] while adding per-device adaptivity and closed-loop enforcement. Each objective below corresponds to one stage.

1. **In-kernel eBPF capture (Layer 1).** Design an eBPF program at the kernel/driver boundary that intercepts URBs and extracts multi-modal features — event timing, traffic volume, and descriptor changes — passing compact records upward via shared BPF maps with minimal overhead.
2. **In-kernel anomaly scoring (Layer 2).** Construct a *per-device adaptive baseline* and score incoming features against it using unsupervised learning (Isolation Forest / One-Class SVM) in the kernel fast-path — a fast, high-recall filter that flags suspicious activity for deeper inspection.
3. **Userspace temporal sequence modelling (Layer 3).** Develop a PyTorch LSTM that analyses rolling event windows to detect *temporal structure* — staged, multi-step attacks whose ordering, not any single event, betrays intent — complementing Layer 2's point-in-time detection.
4. **Closed-loop kernel-level enforcement (Layer 4).** Execute verdicts from Layers 2–3 — rate-limiting or unbinding a device — *inside the kernel* with no userspace round-trip, closing the activation-delay window that lets short BadUSB payloads succeed.
5. **Supporting data store and interface.** Persist per-device baselines, event history, and alerts in SQLite, and expose them through a dashboard (FastAPI/Flask) for monitoring, sensitivity tuning, and reviewing or overriding enforcement.
6. **Evaluation against BadUSB attack scenarios.** Evaluate against attacks emulated with low-cost hardware (Raspberry Pi Pico / Digispark) — rapid, delayed, and multi-stage payloads — measuring detection effectiveness, false-positive rate, enforcement latency, and throughput overhead, with behavioural trust compared directly against identity-based baselines.

One decision remains open, to be resolved against the project timeline: Layer 1 capture via eBPF (libbpf/BCC), the richer option, or usbmon as a lighter fallback. The design is agnostic to this choice — it affects only Layer 1.

5. References

[1] X. Shen, N. Li, C. Liu, and M. Cai, "WMUD: A Whitelist Method of USB Devices Redirection for Virtual Desktop," in 2019 IEEE 21st Int. Conf. on High Performance Computing and Communications; IEEE 17th Int. Conf. on Smart City; IEEE 5th Int. Conf. on Data Science and Systems (HPCC/SmartCity/DSS), 2019, pp. 527–533, doi: 10.1109/HPCC/SmartCity/DSS.2019.00083.

[2] R. M. G. S. Bintang, Arizal, R. Purwoko, M. Syahral, S. S. Zen, I. G. M. Putra, and R. A. Wijayanti, "SafeUSB: Implementation of Keystroke Monitoring System to Prevent BadUSB HID Injection Attack," in 2025 IEEE 2nd Int. Conf. on Cryptography, Informatics, and Cybersecurity (ICoCICs), 2025, pp. 286–291, doi: 10.1109/ICOCICS68032.2025.11383963.

[3] S. Kostopoulos, D. Papatsaroucha, I. Kefaloukos, and E. K. Markakis, "eIDPS: A real-time eBPF-based and Machine Learning-powered Network Intrusion Detection and Prevention Solution," in 2025 6th Int. Conf. in Electronic Engineering & Information Technology (EEITE), 2025, doi: 10.1109/EEITE65381.2025.11165804.

[4] R. Rajurkar and N. Deshmukh, "From Packets to Defense: Architectural Patterns in eBPF-Based Intrusion Detection," in 2025 Int. Conf. on Intelligent Computing and Next Generation Networks (ICNGN), 2025, doi: 10.1109/ICNGN67480.2025.11413805.

[5] L. Song, “eBPF: Pioneering Kernel Programmability and System Observability – Past, Present, and Future Insights,” in 2024 3rd Int. Conf. on Artificial Intelligence and Computer Information Technology (AICIT), 2024, doi: 10.1109/AICIT62434.2024.10730620.

[6] M. S. Baek and Y.-A. Jung, “A Two-Stage Deep Learning-based Framework for High-Accuracy Network Intrusion Detection,” in 2025 16th Int. Conf. on Information and Communication Technology Convergence (ICTC), 2025, doi: 10.1109/ICTC66702.2025.11388157.

[7] V. Hnamte, H. Nhung-Nguyen, J. Hussain, and Y. Hwa-Kim, “A Novel Two-Stage Deep Learning Model for Network Intrusion Detection: LSTM-AE,” IEEE Access, vol. 11, pp. 37131–37148, 2023, doi: 10.1109/ACCESS.2023.3266979.

[8] A. Saini, T. K. Jaiswal, Parteek, A. Thakur, A. Rai, and S. Saini, “Hybrid CNN–LSTM Architecture for Cybersecurity Threat Detection in Network Traffic,” in 2026 Int. Conf. on Signal, Systems, and Computing for Next-Gen Automation (ICSSCNA), 2026, doi: 10.1109/ICSSCNA68616.2026.11546877.

[9] M. Chithik Raja and M. M. B. Al Mahri, “Behavioral Anomaly Detection in Big Data Streams: An Attention-Enhanced LSTM Approach for Privacy-Aware Zero-Day Attack Detection,” in 2025 7th Int. Conf. on Innovative Data Communication Technologies and Application (ICIDCA), 2025, doi: 10.1109/ICIDCA66325.2025.11280566.

[10] A. Mohamed Anwar, D. Vikram, Thiraviaselvi G, and S. Ganapathy, “LSTM-Powered Anomaly Detection for Securing IIoT Systems Against DDoS, Spoofing, and Ransomware Attacks,” in 2025 Int. Conf. on Emerging Research in Computational Science (ICERCS), 2025, doi: 10.1109/ICERCS65898.2025.11580832.